From successful probes to verified application delivery

Ashikuzzaman | Geneva pilot, endpoint prototype and proposed research direction
60 final local trials: 24 echo + 24 HTTP comparison + 12 controlled HTTP faults. AI-assisted code and analysis.

Understanding Geneva and the research question

Geneva uses genetic search to construct packet-manipulation strategies; the engine executes them, and application plugins provide fitness feedback. [1] This study uses its engine, actions and evaluator, with two manually selected split strategies; it does not reproduce genetic discovery. My question is: does a strategy that succeeds in one workload preserve correct delivery in another, and what can endpoint evidence establish when it fails?

Security reasoning and implemented contribution

Using existing actions and two unmodified mock censors, I examined TCP stream boundaries, sequence/ACK behavior, reset injection versus packet dropping, and application integrity. I corrected local echo measurement/scoring assumptions and added a `filecheck` plugin. It transfers a deterministic 256 KiB object over local HTTP/1.1, checks status, exact length and SHA-256, and reports endpoint symptoms with evidence. Censor logs are evaluation ground truth only. The earlier echo pilot, new prototype and all final results are retained.

Observed application completion: echo versus HTTP file

	Echo Drop	Echo Reset	HTTP file Drop	HTTP file Reset
Benign / no strategy	3/3	3/3	3/3	3/3
Forbidden / no strategy	0/3	0/3	0/3	0/3
Forbidden / split22	0/3	3/3	0/3	0/3
Forbidden / split26	3/3	3/3	3/3	3/3

Each cell: 3 local trials. Echo: 5 exact exchanges. HTTP: status 200 + exact 256 KiB reference file.

Main observation: split22 completed 3/3 echo trials against the reset censor but 0/3 HTTP file trials. Split26 completed 3/3 in both workloads and both censors, with no keyword-trigger entries. This is workload-conditioned evidence, not a real-world reliability estimate.

Why the split22 contrast is informative

In echo, the keyword is detected, but returning the first 22 bytes can advance the client stream before injected resets arrive. Across the three final echo captures, all 75 observed resets were 22 bytes behind previously observed server data. In the three HTTP captures, the first reset arrived before any server payload; its sequence equaled the latest client ACK, its TCP checksum validated, and the client reported reset at headers. The evidence is consistent with reset timing/sequence validation [2], but does not directly reveal kernel state. Workloads also differ in framing, size and exchange count, so application semantics alone are not causally isolated.

Controlled fault validation: more than a successful status code

Each of four benign-request faults was run three times: one-byte corruption, truncation, HTTP 503 and delayed response. All 12 were rejected with the expected observable symptom. On this constructed fault set, a naive HTTP-200-only rule falsely accepts 6/12, and HTTP-200-plus-length accepts 3/12; exact-object verification accepts 0/12. These are sanity checks for delivery criteria, not comparisons to Geneva's full HTTP plugin or learned root-cause accuracy.

Proposed direction: application-aware strategy validation

What is implemented, and what remains open

The current endpoint prototype verifies delivery and reports an evidence-based symptom. It deliberately does not infer censorship or the root cause of failure. Its rules never receive censor identity, trigger logs, fault mode or a strategy label. All failures return cause `undetermined`. Consequently, lack of false censorship claims is abstention by design, not demonstrated causal discrimination. Error staging is still coarse: a premature HTTP close is reported as a generic protocol/transport error.

I propose extending this prototype into a workload-aware validation study: identify when a strategy's apparent success transfers across applications, which observables explain a failure symptom, and where the evidence remains ambiguous. My interest in explainable network-security models motivates transparent measurements and explicit uncertainty. This proposal builds on the completed prototype; it does not require claiming a learned classifier or an automatic recovery system.

A bounded next experiment

1. Isolate timing. Hold request bytes, response object and segmentation constant while varying only when the controlled server starts sending. Randomize condition order and repeat on a second kernel. Record reset sequence offsets and full-object delivery, rather than equating detection with blocking.

2. Test workload transfer. Predefine a small workload matrix and full-delivery criteria, including control cases and held-out payload sizes. Compare strategy outcomes, detection and endpoint symptoms separately. Retain benign server faults to test whether the observations actually distinguish causes or should remain ambiguous.

3. Evaluate added evidence. Compare a single failed probe with matched benign/keyword-bearing probes under the same strategy and probe budget. Measure verified-delivery errors, unresolved cases and evidence gained per probe. Keep fixed split26 as a baseline: it currently succeeds in both models. Any later recovery or adaptive selection would be separate future work, not this pilot's contribution.

Potential lab value and focused feedback request

The concrete artifact is a new validation workload, endpoint telemetry and an auditable experimental pipeline. Its potential lab value is testing whether successful strategies preserve the application behavior that matters, while making failures inspectable. Does this workload-aware validation direction seem meaningful for Geneva? Which application or deployment scenario should I study next, and what related work or assumptions should I address?

Method, limitations and attribution

Upstream commit `61a4a5104641ddbba6dd0526013e85a5be875203`. User-executed trials: Ubuntu 24.04 VM, Linux 7.0.0-38-generic, Python 3.6.14, Docker Engine 29.1.3. The earlier echo/API/readiness patch is preserved; filecheck is additional code. No censor/action logic was altered. Raw endpoint JSON and client/server/censor logs were cross-checked; packet fields and checksums were independently parsed. Experiments were not independently rerun during audit.

Only three repeats per condition, two simple mock censors, one VM/kernel, fixed condition order and one HTTP object were studied. Application differences are confounded; endpoint cause identification, adaptive selection, natural concept drift, HTTPS and field deployment have not been demonstrated. Hash verification is an established technique; no novelty is claimed without a related-work review. OONI already treats many anomalies as inconclusive. [3] Original Geneva is credited to its authors; implementation, debugging and drafting here were AI-assisted and require applicant review.

References

[1] Bock et al., *Geneva: Evolving Censorship Evasion Strategies*, CCS 2019; geneva.cs.umd.edu/papers/geneva_ccs19.pdf. Plugin/fitness documentation: geneva.readthedocs.io/en/latest/howitworks/evaluation.html
[2] RFC 5961, section 3.2: rfc-editor.org/rfc/rfc5961.html#section-3.2
[3] OONI, *Interpreting OONI data*: ooni.org/support/interpreting-ooni-data/